

DOCUMENTACION TECNICA

Firmware, binarios y flasheo

ESP32-S3 Zero

FIRMWARE-BINARIOS-Y-FLASHEO.MD

Firmware, binarios y flasheo — ESP32-S3 Zero

TABLA DE CONTENIDOS

Firmware, binarios y flasheo — ESP32-S3 Zero	2
Descripcion general.....	2
Los binarios del build.....	3
firmware.bin vs binario merged	3
Reconstruir el firmware desde cero, partiendo del fork.....	4
Por que el respaldo del .bin sigue haciendo falta.....	4
Respaldos generados	5
⚠ El firmware que corre hoy (2026-10-09) NO tiene respaldo empaquetado	5
✓ Con barge-in (2026-09-24) — ultimo respaldo empaquetado	5
Version Hermes bridge del 2026-07-18 (historica, SIN barge-in).....	6
Version Elato cloud (anterior — historica)	6
Placa nueva (unidad soldada).....	6
Como flashear	7
Opcion A — desde el proyecto (lo normal)	7
Opcion B — grabar el binario merged (respaldo, de un tiro).....	7
Opcion C — grabar solo la app	7
Como regenerar los respaldos	8
Solucion de problemas.....	9
Referencia rapida	10

Descripcion general

Guia sobre los **binarios** que produce la compilacion del firmware de ElatoAI para la **Waveshare ESP32-S3 Zero**, que representa cada uno, en que direccion (offset) de la flash va, y como flashear la placa (incluido el **binario merged** de respaldo que se graba de una sola vez).

NOTA El USB del Zero es nativo (USB-JTAG/serial). Por eso todo flasheo con esptool en esta placa usa `--no-stub` (ya configurado en `platformio.ini`). Sin eso, la subida se corta a mitad. Ver `[[port-esp32-s3-zero]]`.

Los binarios del build

Tras `pio run`, los binarios quedan en `firmware-arduino/.pio/build/esp32-s3-zero/`:

Binario	Offset flash	Tamaño aprox	Que es
<code>bootloader.bin</code>	0x0	~15 KB	Segundo gestor de arranque (en el S3 va en 0x0, no en 0x1000)
<code>partitions.bin</code>	0x8000	~3 KB	Tabla de particiones (de <code>partition.csv</code> , 4 MB)
<code>boot_app0.bin</code>	0xe000	8 KB	Selector OTA (indica que app arrancar). Viene del framework
<code>firmware.bin</code>	0x10000	~1.2 MB	La aplicación (tu firmware)

NOTA (S3): A diferencia del ESP32 clásico (bootloader en 0x1000), en el **ESP32-S3 el bootloader va en 0x0**. El merged de este doc ya lo coloca bien.

`firmware.bin` vs binario merged

Hay dos formas de guardar/flashear el firmware. Es la duda más común:

```

¿Que quieres hacer?
/      \
Actualizar solo      Reinstalar TODO
la app              desde cero / respaldo
|                  |
▼                  ▼
firmware.bin        firmware_merged.bin
-> offset 0x10000    -> offset 0x0
(necesita que el     (incluye bootloader +
bootloader+parts     particiones + boot_app0
ya estén en la placa) + app, todo en uno)

```

	<code>firmware.bin</code> (solo app)	<code>..._merged.bin</code> (todo)
Contiene	Solo la aplicacion	bootloader + particiones + boot_app0 + app
Offset de grabado	<code>0x10000</code>	<code>0x0</code>
Cuando usar	Actualizar la app en una placa ya inicializada	Reinstalar desde cero, clonar, o respaldo
Sirve solo	No para primer flasheo	Si, deja la placa lista de un tiro

TIP Para un **respaldo** o para reinstalar todo con un solo comando, usa el **merged**. Para el día a día (recompilar y subir), PlatformIO ya graba los 4 binarios en sus offsets automaticamente con `pio run -t upload`.

Reconstruir el firmware desde cero, partiendo del fork

El repositorio tiene **todo lo necesario para recompilar**: las fuentes, el `platformio.ini` con sus dependencias, la tabla de particiones y hasta el `startup.mp3`. Y la IP del servidor esta **parametrizada**, asi que no hay que editar ningun archivo:

```
git clone -b feature/esp32-s3-zero-port https://github.com/cag446/ElatoAI
cd ElatoAI/firmware-arduino
HERMES_SERVER_IP=192.168.100.23 pio run -t upload --upload-port /dev/ttyACM0
```

La variable la consume `scripts/hermes_server_ip.py`, que la inyecta como `-D HERMES_SERVER_IP` en el build (ver el comentario de `src/Config.cpp:45`).

Por que el respaldo del `.bin` sigue haciendo falta

Las dependencias estan fijadas con `^` — por ejemplo `links2004/WebSockets@^2.4.1` —, que permite actualizaciones **menores**. Si dentro de un año una de esas librerias cambia algo, `pio run` va a traer una version distinta de la que se uso hoy y **el binario resultante puede no ser identico** al que esta funcionando.

Por eso el respaldo con el `.bin` ya compilado no es redundante con el codigo fuente: el repositorio te da la **receta**, el respaldo te da el **resultado que sabes que funciona**. Ante un problema raro despues de recompilar, comparar contra el binario del respaldo es la forma mas rapida de saber si la causa fue un cambio propio o una dependencia que se movio sola.

Si en algun momento hace falta reproducir el build **exactamente**, hay que cambiar los `^` por versiones exactas en `platformio.ini`.

Respaldos generados

En el home del usuario hay varios juegos de respaldos. **No son intercambiables.**

⚠ El firmware que corre hoy (2026-10-09) NO tiene respaldo empaquetado

Desde el 2026-10-08 el parlante corre un firmware **posterior** al respaldo del 09-24: agrega el recuadro "Clave del bridge" en el portal (`POST /api/wifi/token`) y baja `CORE_DEBUG_LEVEL` a 1 (commits `3b3846c` y `a170fc5`). Se reconstruye desde el fork con la Opcion A. Falta regenerar el paquete de respaldo con este binario.

Antes de restaurar el respaldo del 09-24, leer esto:

- Su binario merged cubre `0x0` — `0x3F0000` y trae la zona NVS (`0x9000` — `0xE000`) **en blanco** (verificado 2026-10-09): grabarlo **borra la WiFi y la clave** guardadas en la placa.
- Ese firmware no tiene el recuadro de la clave: con la NVS vacia pide clave a `GET :3000/api/generate_auth_token` , que en modo `enforce` da 404. **El parlante no podria conectarse.** Para usarlo hay que poner antes el bridge en `BRIDGE_AUTH_MODE=transition` (ver `claves-y-tareas-largas.md`).

Respaldo de la flash **completa** tal como estaba antes del flasheo del 2026-10-08 (incluye NVS con la WiFi y el JWT viejo de Elato; no copiarlo a otros lados): `~/elatoai-flash-previo-2026-10-08_210441/flash-completo.bin` (4 MB, `SHA256SUMS` al lado). Grabarlo en `0x0` deja la placa exactamente como estaba ese dia.

✓ Con barge-in (2026-09-24) — ultimo respaldo empaquetado

`~/elatoai-firmware-respaldo-2026-09-24.{tar.gz,zip}` , y copia fuera de la VM en la Mac mini (`~/Respaldos/elatoai/`).

Paquete autocontenido: `LEEME.md` , `flash.sh` (graba el merged en 0x0 con `--no-stub`), `verificar.sh` + `SHA256SUMS` , los cinco binarios con sus offsets, y `docs/` . Binario compilado el **2026-09-21 20:46**, con `AEC_ENABLED=1` (`AEC_FRAME=64` , `AEC_FILTER=512`) y el arreglo del carraspeo.

Incluye las **Fases A, B y C**: barge-in por boton y por voz.

⚠ **Los respaldos que siguen son ANTERIORES AL BARGE-IN.** Restaurarlos hace perder el corte por boton y por voz. Se conservan como historico.

Version Hermes bridge del 2026-07-18 (historica, SIN barge-in)

Firmware que habla con `bridge.py` en la Mac Mini (`$HERMES_SERVER_IP:8000`), ver `[[runbook-hermes-bridge]]`, pipeline local: VAD -> Whisper -> Hermes -> Piper. Sin nube.

Archivo	Que es	Offset	MD5
<code>~/ElatoAI_ESP32_S3_Bridge.py.bin</code>	Solo la app	0x10000	<code>075e982cc43c1a370a35ec61e07ba28c</code>
<code>~/ElatoAI_ESP32_S3_Bridge.py_merged.bin</code>	Todo junto	0x0	<code>f2aede4cb8f14dddeda29cff684134ba</code>

- **Origen:** build del 2026-07-18 23:19, rama `feature/esp32-s3-zero-port`, commit `6bb603b`. Es la version de produccion validada (ver `runbook-hermes-bridge.md`).
- **Exportados:** 2026-08-03.

Version Elato cloud (anterior — historica)

Firmware original que hablaba con la nube de ElatoAI. Se conserva por si hace falta volver atras.

Archivo	Que es	Offset
<code>~/Firmware_ELATO_ok_limpio.bin</code>	Solo la app	0x10000
<code>~/Firmware_ELATO_ok_limpio_merged.bin</code>	Todo junto	0x0

- **Origen:** build del 2026-07-18 18:20 (anterior a los cambios del bridge).

COMO DISTINGUIRLOS: el nombre `Bridge.py` indica el firmware que habla con `bridge.py` (local). El `ELATO` es el de la nube. Ante la duda, usar el **Bridge.py**, que es lo que corre hoy en la placa.

Placa nueva (unidad soldada)

Para una placa **virgen** —el caso de la unidad definitiva con los componentes soldados— hay que grabar el **merged a** `0x0`, porque una placa de fabrica no trae bootloader ni tabla de particiones:

```
esptool --chip esp32s3 --no-stub --port /dev/ttyACM0 write-flash \  
0x0 ~/ElatoAI_ESP32_S3_Bridge.py_merged.bin
```

Despues de flashear, la placa necesita configuracion de red igual que el prototipo. El conexionado de los componentes esta en `conexionado-por-componente.md` (mic, parlante, boton, LED).

ANTES DE SOLDAR: verificar contra `conexionado-por-componente.md` que los pines coinciden con los que espera este firmware. El binario esta compilado con un pinout fijo — si la placa soldada cablea distinto, hay que recompilar, no basta con reflashear.

Como flashear

Opcion A — desde el proyecto (lo normal)

Recompila y graba los 4 binarios en sus offsets automaticamente:

```
cd firmware-arduino
HERMES_SERVER_IP=192.168.100.23 pio run -t upload --upload-port /dev/ttyACM0
```

No toca la NVS: la WiFi y la clave del bridge siguen guardadas. Despues del upload el Zero suele **quedar en modo descarga** (cada reset por USB lo vuelve a dejar ahi): desenchufar y enchufar **sin tocar BOOT** para que arranque.

Opcion B — grabar el binario merged (respaldo, de un tiro)

Graba TODO a offset `0x0` con esptool. **Es la opcion para una placa nueva.**

```
esptool --chip esp32s3 --no-stub --port /dev/ttyACM0 write-flash \
0x0 ~/ElatoAI_ESP32_S3_Bridge.py_merged.bin
```

Opcion C — grabar solo la app

Solo si el bootloader y las particiones ya estan en la placa:

```
esptool --chip esp32s3 --no-stub --port /dev/ttyACM0 write-flash \
0x10000 ~/ElatoAI_ESP32_S3_Bridge.py.bin
```

ADVERTENCIA Si el Zero no entra en modo descarga, **manten pulsado BOOT (GPIO0) mientras conectas el USB-C**, suelta, y reintenta. Tras grabar, la placa se reinicia y el puerto USB-CDC se re-enumera (es normal que `/dev/ttyACM0` desaparezca y vuelva).

Como regenerar los respaldos

Si recompilas y quieres exportar un juego nuevo. Los dos pasos, tal como se generaron los respaldos actuales (verificado 2026-08-03):

1. Copiar la app sola:

```
cp firmware-arduino/.pio/build/esp32-s3-zero/firmware.bin \
~/ElatoAI_ESP32_S3_Bridge.py.bin
```

2. Generar el merged:

```
cd firmware-arduino/.pio/build/esp32-s3-zero
BOOTAPP0=./.platformio/packages/framework-arduinoespressif32/tools/partitions/boot_app0.bin
esptool --chip esp32s3 merge-bin -o ~/ElatoAI_ESP32_S3_Bridge.py_merged.bin \
--flash-mode dio --flash-freq 80m --flash-size 4MB \
0x0 bootloader.bin \
0x8000 partitions.bin \
0xe000 "$BOOTAPP0" \
0x10000 firmware.bin
```

Salida esperada: `Wrote 0x13c6f0 bytes ... ready to flash to offset 0x0`.

NOTA TIP: Verifica que la copia es identica al original con `md5sum`, y anota el nuevo MD5 en la tabla de respaldos de este documento.

`merge-bin` NO necesita la placa conectada — solo junta archivos. El `--no-stub` hace falta al **grabar**, no al generar el merged.

Solucion de problemas

Sintoma	Causa probable	Solucion
<code>Packet content transfer stopped</code> al subir	Falta <code>--no-stub</code> (USB nativo del Zero)	Ya esta en <code>platformio.ini</code> ; si usas esptool a mano, anade <code>--no-stub</code>
<code>Could not open /dev/ttyACM0</code>	El puerto se re-enumero tras un upload	Reconecta el USB-C y reintenta
Tras grabar, el log serie muestra <code>boot:0x0 (DOWNLOAD(USB/UART0))</code>	El reset por USB del Zero lo deja en modo descarga	Desenchufar y enchufar sin BOOT
<code>read_flash</code> con <code>--no-stub</code> falla pasados los 2 MB (<code>CRC or checksum was invalid</code>)	Sin stub, la ROM asume flash de 2 MB	Leer con la libreria de esptool llamando antes <code>flash_set_parameters(<tamaño real>)</code> . Paso con el Cardputer (8 MB) el 2026-10-08; en el Zero (4 MB) <code>read_flash 0 0x400000</code> anduvo directo
<code>read_flash</code> sin <code>--no-stub</code> falla con <code>Corrupt data</code>	El stub sobre USB nativo	Usar <code>--no-stub</code>
<code>No module named 'rich_click'</code> con <code>esptool.py</code>	Esa copia de esptool no tiene deps	Usa el comando <code>esptool</code> (v5.x) instalado por separado
La placa arranca en bucle tras grabar solo <code>firmware.bin</code>	Bootloader/ particiones ausentes o incompatibles	Graba el merged a <code>0x0</code>
Bootloader no arranca (S3)	Se grabo el bootloader en <code>0x1000</code>	En el S3 el bootloader va en 0x0

Referencia rapida

OFFSETS (ESP32-S3):

```
0x0      bootloader.bin
0x8000   partitions.bin
0xe000   boot_app0.bin
0x10000  firmware.bin      <- la app
```

RESPALDOS (~/):

```
[Flash completa previa al 2026-10-08 – incluye NVS]
elatoai-flash-previo-2026-10-08_210441/flash-completo.bin -> todo, offset 0x0
[Con barge-in, 2026-09-24 – ultimo paquete; SIN clave del bridge]
elatoai-firmware-respaldo-2026-09-24/    -> flash.sh (merged a 0x0, BORRA NVS)
[Hermes bridge 2026-07-18 – historica, SIN barge-in]
ElatoAI_ESP32_S3_Bridge.py.bin           -> app sola, offset 0x10000
ElatoAI_ESP32_S3_Bridge.py_merged.bin    -> todo, offset 0x0
[Elato cloud – historica]
Firmware_ELATO_ok_limpio.bin              -> app sola, offset 0x10000
Firmware_ELATO_ok_limpio_merged.bin       -> todo, offset 0x0
```

FLASHEAR:

```
pio run -t upload                # normal (proyecto)
esptool --chip esp32s3 write-flash 0x0 <merged>  # placa nueva / respaldo
esptool --chip esp32s3 write-flash 0x10000 <app>  # solo app
```

SIEMPRE en el Zero: --no-stub (USB nativo). Modo descarga: BOOT + conectar USB-C.

Tras grabar: desenchufar y enchufar SIN BOOT para que arranque.

Firmware de HOY (con clave del bridge): reconstruir desde el fork (Opcion A).

OJO: el merged del 09-24 borra la NVS (WiFi + clave) y no funciona con el bridge en modo enforce.